Thực nghiệm giải N-Queens bằng SAT, CP và MIP đến N=1000

Bùi Đình Cảnh[a]

[a]*Sinh viên 24021392, lớp INT3011E_3, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội; email: buidinhcanh2006@gmail.com*

ARTICLE INFO

Keywords:
N-Queens
SAT solving
cardinality encoding
PySAT
constraint programming
mixed-integer programming

Abstract

Bài báo nghiên cứu các mô hình giải chính xác cho bài toán N-Queens, với trọng tâm là ảnh hưởng của kỹ thuật mã hóa ràng buộc at-most-one trong SAT. Năm mã hóa Pairwise, Sequential Counter, Commander, Product và Binary được triển khai trên PySAT/Glucose3 và so sánh với OR-Tools CP-SAT, Gurobi MIP, CPLEX MIP và CPLEX CP Optimizer. Quy trình thực nghiệm có timeout cưỡng bức, checkpoint, kiểm tra nghiệm và phân biệt timeout, giới hạn giấy phép và tiến trình bị hệ điều hành dừng. Bộ dữ liệu sau khi bổ sung Gurobi Academic WLS gồm 241 lượt chạy trên các kích thước từ $N = 1$ đến $N = 1000$. Tất cả 182 nghiệm SAT thu được đều vượt qua bộ kiểm tra độc lập; 14 lượt UNSAT tại $N = 2, 3$ cũng phù hợp với tính chất bài toán. Trên môi trường Google Colab đã sử dụng, Gurobi hoàn thành 5/5 lượt tại $N = 64, 128$ và 3/3 lượt tại $N = 256$. Product là mã hóa SAT duy nhất hoàn thành cả ba lần tại $N = 1000$, với trung vị 28.127 giây, 1,298,508 biến và 11,942,164 mệnh đề. Kết quả cũng cho thấy kích thước CNF không đủ để dự đoán thời gian giải: hiệu năng thay đổi không đơn điệu theo N và phụ thuộc mạnh vào cấu trúc encoding cũng như heuristic của solver.

1. Giới thiệu

Bài toán N-Queens yêu cầu đặt N quân hậu trên bàn cờ có N hàng và N cột sao cho không có hai quân nào cùng hàng, cùng cột hoặc cùng đường chéo. Bài toán có mô tả ngắn nhưng là một ví dụ điển hình để khảo sát ảnh hưởng của mô hình hóa tới hiệu năng solver. Cùng một tập ràng buộc có thể được biểu diễn bằng công thức SAT ở dạng CNF, bằng các ràng buộc toàn cục trong Constraint Programming (CP), hoặc bằng hệ ràng buộc tuyến tính trên biến nhị phân trong MIP.

Sự khác biệt giữa ba biểu diễn nằm ở cấu trúc mà solver có thể khai thác. SAT yêu cầu chuyển ràng buộc đếm thành CNF và có thể tạo nhiều biến phụ; CP giữ được ràng buộc `AllDifferent`; MIP dùng biến nhị phân cùng các bất đẳng thức tuyến tính. Bài toán vì thế phù hợp để khảo sát chi phí của mô hình hóa tách biệt với việc tìm một nghiệm khả thi.

Mục tiêu của nghiên cứu không phải đếm mọi lời giải, mà tìm một cấu hình hợp lệ hoặc chứng minh không tồn tại cấu hình. Đóng góp chính gồm: (i) triển khai và kiểm thử năm SAT encoding cho AMO; (ii) xây dựng bốn mô hình đối chứng bằng OR-Tools, Gurobi và CPLEX; (iii) xây dựng pipeline thực nghiệm có thể tiếp tục sau khi Colab bị ngắt; và (iv) đánh giá đồng thời thời gian, quy mô encoding và tỷ lệ hoàn thành đến $N = 1000$.

2. Bối cảnh và câu hỏi nghiên cứu

Các solver SAT hiện đại kết hợp tìm kiếm CDCL và học mệnh đề; hiệu năng của chúng phụ thuộc vào công thức CNF được cung cấp [1]. Đối với ràng buộc at-most-one (AMO), Pairwise ngắn gọn và không dùng biến phụ, trong khi Sequential Counter dùng biến trạng thái để giảm số clause [8]. PySAT cung cấp nhiều kiểu cardinality encoding với cùng giao diện [6, 7]. N-Queens cũng có mô hình CP tự nhiên với ba ràng buộc `AllDifferent` [3].

Báo cáo trả lời ba câu hỏi: (i) kích thước CNF của năm cách mã hóa AMO thay đổi như thế nào theo N; (ii) sự thay đổi đó có dự báo được thời gian giải hay không; và (iii) SAT, CP-SAT và MIP khác nhau ra sao về tỷ lệ hoàn thành trong cùng giới hạn thời gian. So sánh thực nghiệm được giới hạn bởi những giá trị N đã chạy được và điều kiện cấp phép của từng solver.

ORCID(S):

Table 1
Bậc kích thước của một nhóm AMO gồm k literal. Các hằng số và trường hợp nhóm rất nhỏ phụ thuộc cài đặt cụ thể.

Mã hóa	Biến phụ	Clause
Pairwise	0	$O(k^2)$
Sequential	$O(k)$	$O(k)$
Binary	$O(\log k)$	$O(k \log k)$
Commander	$O(\sqrt{k})$	$O(k\sqrt{k})$
Product	$O(\sqrt{k})$	$O(k)$

3. Mô hình bài toán

Với mỗi ô (r, c), đặt biến Boolean $x_{r,c}$ bằng 1 khi có hậu tại ô đó. Mô hình SAT và MIP sử dụng các nhóm ràng buộc sau:

$$\sum_{c=0}^{N-1} x_{r,c} = 1 \qquad \text{với mọi hàng } r, \tag{1}$$

$$\sum_{r=0}^{N-1} x_{r,c} = 1 \qquad \text{với mọi cột } c, \tag{2}$$

$$\sum_{(r,c) \text{ thuộc } d} x_{r,c} \leq 1 \qquad \text{với mọi đường chéo } d. \tag{3}$$

Mỗi ô có đúng một biến cơ sở, nên mô hình SAT/MIP bắt đầu với N^2 biến. Mô hình MIP cài đặt $2N$ ràng buộc hàng/cột và $4N - 2$ ràng buộc đường chéo, trong đó các đường chéo chỉ có một ô tạo bất đẳng thức dư nhưng không làm thay đổi tập nghiệm. SAT chỉ sinh AMO cho đường chéo dài ít nhất hai ô.

Trong SAT, ràng buộc bằng 1 được tách thành at-least-one (ALO) và at-most-one (AMO). Với nhóm literal $X = (x_1, \ldots, x_k)$, ALO là clause $(x_1 \text{ OR } \cdots \text{ OR } x_k)$. AMO được thay đổi theo từng encoding. Trong CP, mỗi hàng i dùng một biến nguyên q_i nhận giá trị từ 0 đến $N - 1$ để biểu diễn cột của quân hậu; mô hình chỉ cần ba ràng buộc toàn cục `AllDifferent` trên q_i, $q_i + i$ và $q_i - i$, đúng với mô hình N-Queens được OR-Tools công bố [3].

Hai mô hình biểu diễn cùng một tập cấu hình: hàng r được gán cột $q_r = c$ đúng khi $x_{r,c} = 1$. `AllDifferent` trên q_r loại xung đột cột; hai biểu thức còn lại lần lượt loại hai hướng đường chéo. Điều này cho phép kiểm tra nghiệm do mọi solver trả về bằng cùng một hàm xác minh độc lập.

4. Các phương pháp giải

4.1. SAT và các mã hóa AMO

PySAT cung cấp giao diện thống nhất cho solver và các encoding cardinality [7, 6]. Glucose3 được dùng làm SAT backend. Pairwise sinh clause $(\overline{x_i} \text{ OR } \overline{x_j})$ cho mọi cặp. Sequential Counter dùng biến trạng thái phụ tuyến tính theo kích thước nhóm, dựa trên ý tưởng của Sinz [8]. Binary gán mã bit cho vị trí của literal đúng. Commander chia các literal thành nhóm và tạo biến "chỉ huy" cho mỗi nhóm. Product ánh xạ literal lên một lưới phụ hai chiều và áp dụng AMO cho các chỉ báo hàng/cột.

Pairwise, Sequential và Binary gọi trực tiếp `CardEnc` của PySAT; Commander và Product được cài đặt trong project. ALO được áp dụng cho mọi hàng và cột; AMO được áp dụng cho hàng, cột và đường chéo. Bộ kiểm thử truth-table kiểm tra trực tiếp rằng cả năm cài đặt AMO chấp nhận đúng các phép gán có không quá một literal đúng. Ngoài ra, Glucose3 là solver CDCL có heuristic học clause; những đặc trưng này giải thích vì sao số clause không ánh xạ đơn giản thành thời gian chạy [1].

Với nhóm X gồm k literal, Pairwise tạo $\binom{k}{2}$ clause nhị phân và không cần biến phụ. Sequential Counter biểu diễn thông tin tiền tố đã có literal đúng bằng biến phụ. Binary gán cho literal đúng một mã dài $\lceil \log_2 k \rceil$ bit. Bảng 1 tóm tắt bậc kích thước của từng nhóm AMO theo đúng cách cài đặt trong project; ALO bổ sung đúng một clause độ dài k cho mỗi hàng/cột.

Trong Commander, k literal được chia thành các nhóm kích thước xấp xỉ $\sqrt{k}$. Mỗi nhóm có một biến chỉ huy c_g; hai chiều kéo theo giữa c_g và phép OR của nhóm bảo đảm c_g đúng chính xác khi nhóm có literal đúng. AMO Pairwise

được áp dụng bên trong mỗi nhóm và giữa các chỉ huy. Product đặt các literal lên lưới phụ $p \times q$ với $p = \lceil \sqrt{k} \rceil$, $q = \lceil k/p \rceil$. Một literal đúng kéo theo hai biến chỉ báo hàng và cột tương ứng; AMO Pairwise trên hai tập chỉ báo ngăn đồng thời hai literal đúng. Với nhóm tối đa bốn literal, bản cài đặt dùng Pairwise trực tiếp.

4.2. Lập luận bảo toàn nghiệm

ALO cùng AMO trên mỗi hàng và cột buộc đúng một hậu tại mỗi hàng/cột. AMO trên hai họ đường chéo loại mọi cặp hậu tấn công nhau. Vì vậy phép gán CNF thỏa mãn luôn cho một bàn cờ hợp lệ. Chiều ngược lại, từ một bàn cờ hợp lệ đặt các biến ô tương ứng bằng đúng và các biến ô còn lại bằng sai; mọi nhóm ALO/AMO đều thỏa. Các biến phụ của Sequential và Binary có thể được gán theo encoding của PySAT. Với Commander, đặt chỉ huy đúng cho nhóm chứa literal đúng. Với Product, đặt đúng chỉ báo hàng/cột chứa literal đúng. Do đó mọi bàn cờ hợp lệ đều mở rộng thành một phép gán CNF thỏa mãn.

Kiểm thử truth-table xét toàn bộ 2^5 phép gán của nhóm năm literal cho cả năm encoding và so sánh kết quả solver với điều kiện nhiều nhất một bit đúng. Kiểm thử đầu-cuối tiếp tục xác nhận SAT cho $N = 1, 4, \ldots, 8$ và UNSAT cho $N = 2, 3$, đồng thời kiểm tra mọi nghiệm bằng hàm độc lập.

4.3. CP-SAT và CP Optimizer

Mô hình OR-Tools CP-SAT và CPLEX CP Optimizer đều dùng N biến nguyên cùng ba ràng buộc `AllDifferent`. CP-SAT là solver CP trên nền SAT cho biến nguyên [2]; CP Optimizer cung cấp các ràng buộc toàn cục cho bài toán tổ hợp [5]. Mỗi solver được giới hạn một worker để giảm sai lệch do mức độ song song khác nhau.

Mô hình CP không cần liệt kê N^2 biến ô. Mỗi $q_r \in \{0, \ldots, N-1\}$ chọn cột cho hàng r; ba ràng buộc `AllDifferent` lần lượt áp dụng cho các dãy (q_r), $(q_r + r)$ và $(q_r - r)$. Cùng một biểu diễn mức cao được đưa cho hai engine CP, song cơ chế lan truyền và tìm kiếm nội bộ khác nhau nên thời gian vẫn có thể khác đáng kể.

4.4. Gurobi MIP và CPLEX MIP

Hai mô hình MIP dùng N^2 biến nhị phân và các phương trình/bất phương trình ở phần trước. Đây là bài toán khả thi, vì vậy không cần hàm mục tiêu. Gurobi và CPLEX được cấu hình một thread, seed theo lần lặp và time limit nội bộ. Gurobi tuân theo mô hình formulate–solve–query của API tối ưu toán học [4]; CPLEX MIP dùng cùng mô hình để tạo phép so sánh giữa hai solver thương mại.

Sau khi solver dừng, nếu có nghiệm khả thi, chương trình lấy các ô có giá trị lớn hơn $0{,}5$ và kiểm tra lại số hậu, miền tọa độ, hàng, cột và đường chéo. Trạng thái hết giờ được phân biệt với vô nghiệm đã chứng minh; lỗi license được ghi thành `LICENSE_LIMIT`, không gán thời gian giải bằng không.

5. Thiết kế thực nghiệm

Thực nghiệm chạy trên Google Colab với 2 CPU logic, một worker cho mỗi solver và bốn phase:

- smoke: $N = 1, 2, 3, 4, 8$, một lần, timeout 15 giây;
- standard: $N = 16, 32, 64, 128$, năm lần, timeout 60 giây;
- CPLEX: $N = 8, 16, 32$, ba lần, timeout 60 giây;
- large: $N = 256, 512, 1000$, ba lần, timeout 120 giây.

Môi trường ghi nhận Linux, Python 3.13.16, PySAT 0.1.8.dev9, OR-Tools 9.15.6755, Gurobi 13.0.3 và CPLEX 22.1.2.1. Pairwise không được chạy ở phase large vì số clause tăng mạnh. Sau đợt đo ban đầu, giấy phép Gurobi Academic WLS đã cho phép chạy lại $N = 64, 128$, năm lần mỗi kích thước, và chạy bổ sung $N = 256$ ba lần. Các file kết quả ban đầu được giữ nguyên; khi gộp, mỗi run Gurobi mới thay thế đúng khóa $(phase, N, method, run)$ cũ bị `LICENSE_LIMIT`. Vì vậy bộ gộp có thêm ba run ở $N = 256$, không nhân đôi mười run tại $N = 64, 128$.

`Total_Time` gồm thời gian xây dựng mô hình/encoding và thời gian giải, nhưng không gồm chi phí khởi tạo process cha. `Wall_Time` bao gồm cả khởi tạo process và IPC. Với Glucose3, seed trong script không được truyền vào solver; vì vậy các lần lặp SAT đo độ dao động thời gian trên cùng cấu hình tìm kiếm mặc định, không phải các lần chạy ngẫu nhiên độc lập.

Mỗi lượt chạy được thực hiện trong process riêng. Process cha đợi tối đa giới hạn thời gian đã chọn, sau đó dừng process con nếu cần; ngay sau mỗi lượt, một dòng CSV được ghi và đồng bộ xuống đĩa để có thể tiếp tục khi Colab

Table 2
Phân bố trạng thái của toàn bộ lượt chạy.

Trạng thái	Số lượt
SAT	182
UNSAT	14
TIMEOUT	39
LICENSE_LIMIT	3
KILLED	3
Tổng	241

ngắt. Chỉ các run có trạng thái SAT hoặc UNSAT mới được dùng để tính trung vị, min và max của `Total_Time`; tỷ lệ thành công được tính trên toàn bộ run, kể cả timeout và lỗi. Không nội suy thời gian cho run chưa hoàn thành.

Các run bổ sung dùng một phiên Colab khác đợt đo gốc. Metadata mới ghi nhận 2 CPU logic và 12,67 GiB RAM, nhưng metadata cũ không lưu model CPU và tổng RAM. Vì vậy các so sánh thời gian giữa Gurobi bổ sung và các solver đã đo trước đó chỉ mang tính mô tả, không phải một phép so sánh phần cứng được kiểm soát chặt.

6. Kết quả và thảo luận

6.1. Tính đúng đắn và mức độ hoàn thành

Bộ dữ liệu có 241 run, không có khóa $(phase, N, method, run)$ bị trùng. Tổng hợp trạng thái được trình bày trong Bảng 2. Tất cả 182 nghiệm SAT đều có `Solution_Valid=True`; 14 kết quả UNSAT đều thuộc $N = 2$ và $N = 3$. Không có `INVALID_SOLUTION`. Ba run `LICENSE_LIMIT` còn lại đều thuộc CPLEX MIP, không thuộc Gurobi.

Hình 1 thể hiện số lần thành công trên tổng số lần chạy. Cần đọc hình này cùng đồ thị thời gian: trung vị chỉ được tính trên các run thành công, nên một điểm thời gian không biểu diễn xác suất hoàn thành.

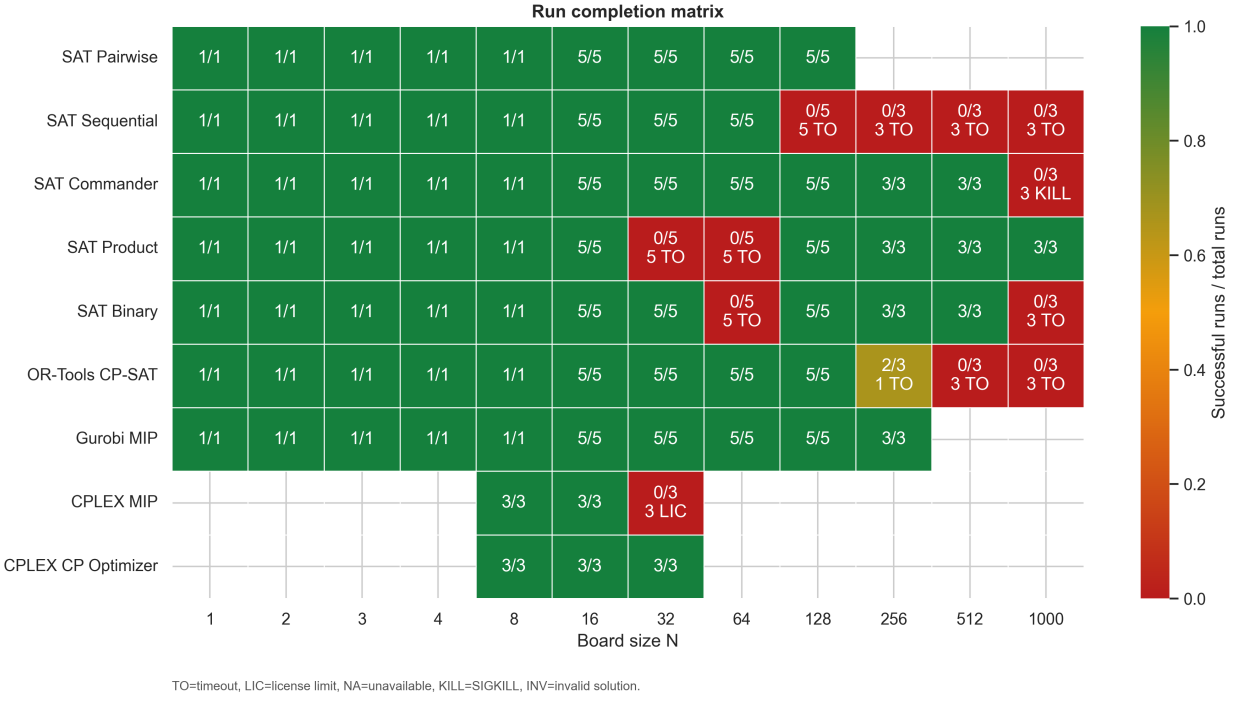


Figure 1: Ma trận hoàn thành. TO: timeout; LIC: giới hạn license; KILL: tiến trình nhận SIGKILL. Ô trống là cấu hình không được đưa vào phase đó.

Table 3
Phương pháp có trung vị thấp nhất trong các cấu hình đã chạy.

N	Phương pháp nhanh nhất	Trung vị (s)	Thành công
16	SAT Sequential	0.0080	5/5
32	Gurobi MIP	0.0103	5/5
64	Gurobi MIP	0.0982	5/5
128	Gurobi MIP	0.1939	5/5
256	SAT Product	2.7036	3/3
512	SAT Binary	13.1581	3/3
1000	SAT Product	28.1271	3/3

6.2. Thời gian giải

Hình 2 và Bảng 3 cho thấy không có một phương pháp thắng ở mọi kích thước. Sequential có trung vị thấp nhất tại $N = 16$; Gurobi MIP tại $N = 32, 64, 128$; Product tại $N = 256, 1000$; Binary tại $N = 512$. Với $N = 64, 128$, Gurobi hoàn thành 5/5 run, trung vị lần lượt 0,098 và 0,194 giây. Tại $N = 256$, Gurobi hoàn thành 3/3 run với trung vị 3,078 giây, còn Product là 2,704 giây. Những thứ hạng này mô tả bộ dữ liệu đã gộp, không khẳng định ưu thế tổng quát giữa các solver.

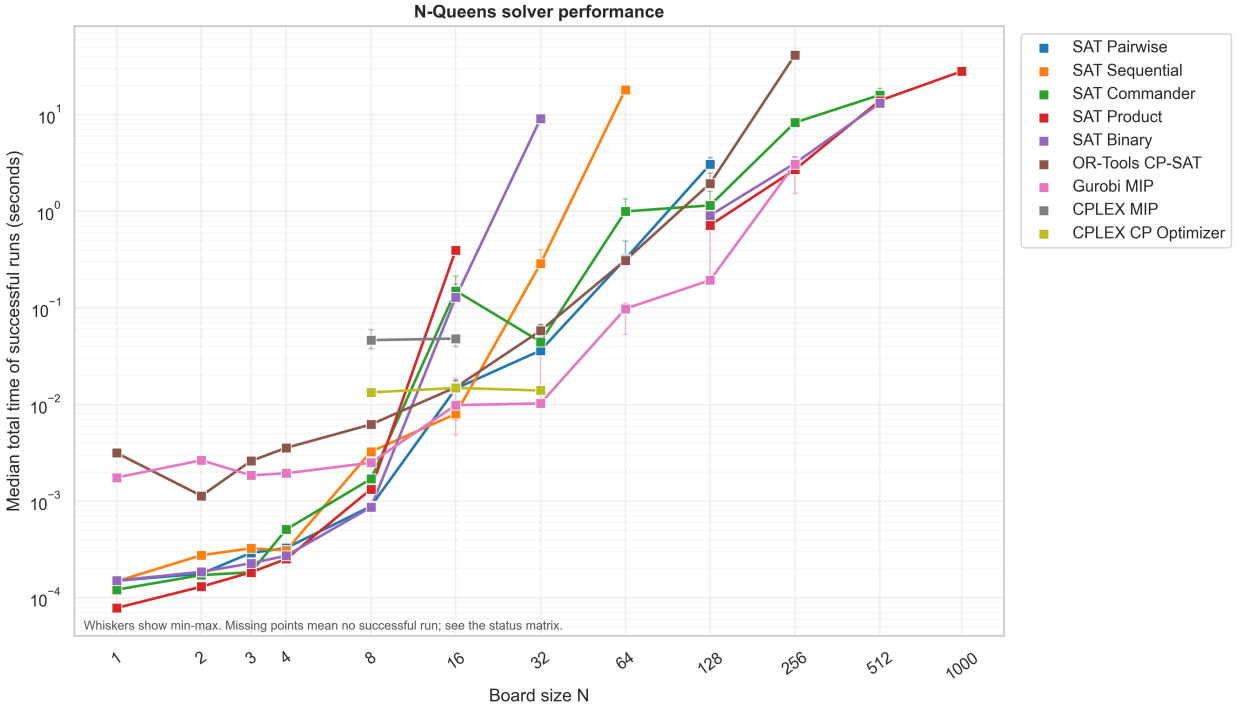


Figure 2: Trung vị thời gian của các run thành công; whisker là min–max. Đường không được nối qua kích thước không có run thành công.

Table 4
Trung vị thời gian theo giây của các run thành công; trong ngoặc là số run thành công/tổng số run. TO: toàn bộ run timeout; dấu gạch ngang: không chạy.

N	Gurobi MIP	CP-SAT	SAT Pairwise	SAT Product	SAT Binary
64	0.098 (5/5)	0.311 (5/5)	0.322 (5/5)	TO	TO
128	0.194 (5/5)	1.944 (5/5)	3.063 (5/5)	0.719 (5/5)	0.908 (5/5)
256	3.078 (3/3)	41.473 (2/3)	–	2.704 (3/3)	3.147 (3/3)

Table 5
Thời gian trung vị SAT theo giây; trong ngoặc là số run thành công/tổng số run. TO: timeout; KILL: SIGKILL; "–": không chạy.

N	Pairwise	Sequential	Commander	Product	Binary
16	0.015 (5/5)	0.008 (5/5)	0.151 (5/5)	0.397 (5/5)	0.129 (5/5)
32	0.036 (5/5)	0.288 (5/5)	0.045 (5/5)	TO	9.090 (5/5)
64	0.322 (5/5)	18.143 (5/5)	0.998 (5/5)	TO	TO
128	3.063 (5/5)	TO	1.153 (5/5)	0.719 (5/5)	0.908 (5/5)
256	–	TO	8.309 (3/3)	2.704 (3/3)	3.147 (3/3)
512	–	TO	16.003 (3/3)	14.004 (3/3)	13.158 (3/3)
1000	–	TO	KILL	28.127 (3/3)	TO

Bảng 4 đặt các run mới cạnh những phương pháp đã chạy trên cùng kích thước. Tại $N = 64$, Product và Binary đều timeout 5/5 lần trong khi Pairwise, CP-SAT và Gurobi hoàn thành. Ở $N = 128$ cả Product và Binary đều trở lại thành công 5/5 lần, nhưng trung vị vẫn lớn hơn Gurobi trong bộ dữ liệu này. Tại $N = 256$, CP-SAT chỉ hoàn thành 2/3 run và Pairwise không được đưa vào phase lớn.

Bảng 5 tập trung vào SAT. Product timeout ở $N = 32, 64$ nhưng hoàn thành cả ba lần ở $N = 1000$; Binary timeout tại $N = 64$ nhưng hoàn thành tại $N = 128, 256, 512$. Đây là bằng chứng thực nghiệm rằng độ khó tìm kiếm không tăng đơn điệu theo N đối với một encoding/heuristic cố định. Không nên diễn giải hiện tượng này thành bảo đảm rằng Product luôn tốt ở kích thước lớn.

OR-Tools hoàn thành 2/3 lần ở $N = 256$ với thời gian thành công khoảng 40–43 giây, sau đó timeout hoàn toàn tại $N = 512, 1000$. CPLEX CP hoàn thành 3/3 lần tại $N = 8, 16, 32$; CPLEX MIP gặp giới hạn license ở $N = 32$. Trong dữ liệu gốc, Gurobi bị giới hạn tại $N = 64, 128$; các kết quả đó đã được thay bằng run Academic WLS hợp lệ. Giới hạn cấp phép CPLEX còn lại phản ánh môi trường, không phải thất bại thuật toán.

Ba run Commander tại $N = 1000$ kết thúc với exit code -9, tức SIGKILL. Điều này phù hợp với tình huống process bị hệ thống Colab dừng do áp lực tài nguyên, nhưng không có log kernel để khẳng định chắc chắn nguyên nhân là OOM; do đó dữ liệu được gắn nhãn thận trọng là `KILLED`.

6.3. Quy mô SAT encoding

Hình 3 và Hình 4 cho thấy Pairwise không dùng biến phụ nhưng số clause tăng nhanh. Product tạo nhiều biến hơn nhưng có số clause thấp hơn rõ rệt ở các kích thước lớn quan sát được. Tại $N = 1000$, Product có 1,298,508 biến và 11,942,164 clause, vẫn hoàn thành với trung vị 28.127 giây. Các run timeout/killed không trả lại thống kê encoding, vì vậy đồ thị dừng đường tại các điểm thiếu thay vì nội suy qua chúng.

7. Hạn chế và nguy cơ ảnh hưởng tính hợp lệ

Thứ nhất, Colab là môi trường chia sẻ; mô hình CPU và tổng RAM chưa được lưu trong bộ metadata cũ, còn Gurobi WLS chạy trong một phiên sau đó. Không nên diễn giải thứ hạng thời gian như một benchmark được kiểm soát phần

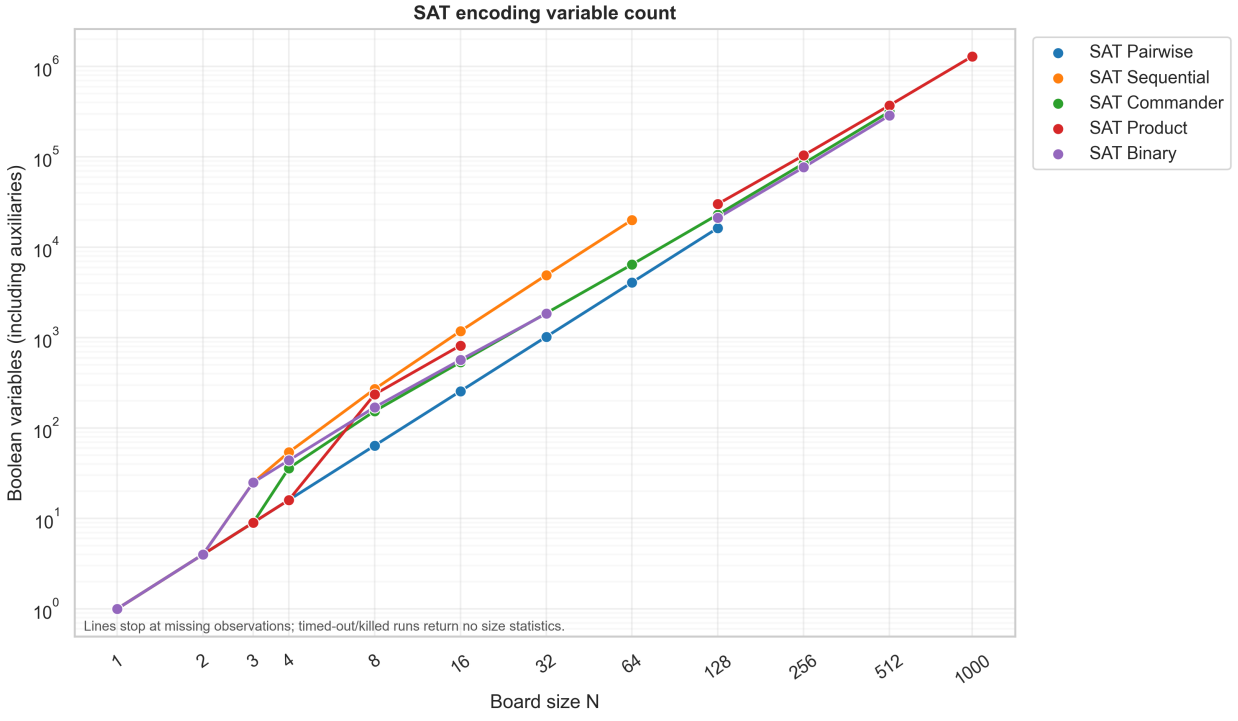


Figure 3: Số biến Boolean, bao gồm biến phụ, của các SAT encoding.

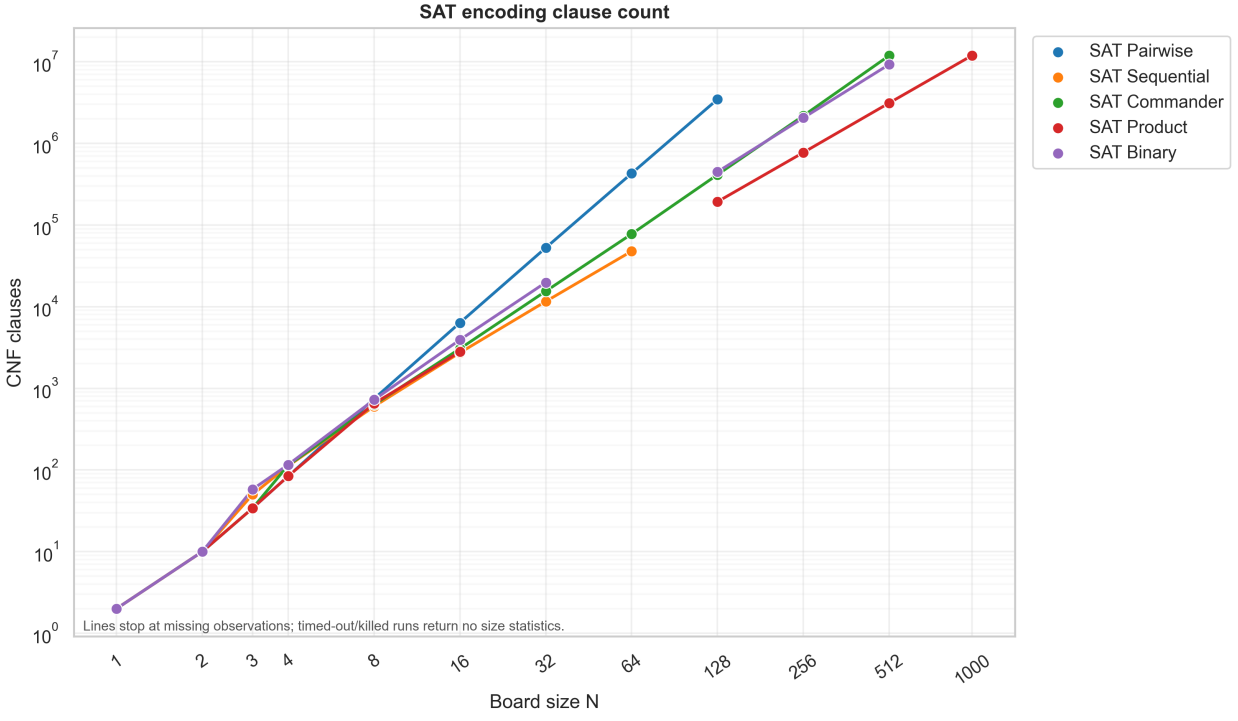


Figure 4: Số clause CNF của các SAT encoding.

cứng. Thứ hai, số lần lặp khác nhau giữa các phase (1, 3 hoặc 5), và SAT không dùng seed ngẫu nhiên. Thứ ba, timeout ở process cha làm mất số biến/clause và không cho biết run dừng trong encoding hay solving. Thứ tư, giới hạn license CPLEX MIP và RAM Colab làm thu hẹp miền kích thước có thể so sánh; Gurobi chỉ được chạy bổ sung đến $N = 256$. Cuối cùng, nghiên cứu chỉ tìm một lời giải khả thi; kết quả không đại diện cho bài toán đếm toàn bộ nghiệm N-Queens.

8. Kết luận

Project đã triển khai đầy đủ các nhóm phương pháp được yêu cầu và xác minh tính đúng đắn trên các trường hợp SAT/UNSAT nhỏ. Bộ dữ liệu 241 run đến $N = 1000$ cho thấy việc chọn encoding có ảnh hưởng quyết định: Pairwise có ít biến nhưng bùng nổ clause, Sequential suy giảm mạnh từ $N = 64$, trong khi Product và Binary mở rộng tốt hơn ở một số kích thước lớn. Gurobi Academic WLS giải thành công toàn bộ run bổ sung tại $N = 64, 128, 256$; song do thực nghiệm qua nhiều phiên Colab, không thể suy rộng thứ hạng thời gian thành ưu thế thuật toán tuyệt đối. Các bất thường không đơn điệu nhấn mạnh rằng không thể đánh giá encoding chỉ bằng số biến hoặc số clause; cần báo cáo đồng thời thời gian, tỷ lệ hoàn thành và giới hạn môi trường.

Data availability

Mã nguồn, unit test, hướng dẫn Colab, dữ liệu thô, bảng tổng hợp và biểu đồ được lưu tại `https://github.com/WilliamsBui-fnd/INT3011E_3`. File `benchmark_raw.csv` được xem là dữ liệu gốc; thư mục `combined_licensed` chứa bản gộp dùng trong báo cáo này, còn các thư mục phase giữ nguyên dữ liệu trước khi thay thế run bị giới hạn license. Mọi bảng và hình trong báo cáo có thể tái tạo bằng `combine_results.py`, `plot_results.py` và `prepare_report.py`.

CRediT authorship contribution statement

Bùi Đình Cảnh: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing.

Declaration of competing interest

Tác giả tuyên bố không có lợi ích tài chính hoặc quan hệ cá nhân có thể ảnh hưởng tới công trình này.

Tuyên bố về công cụ AI hỗ trợ

Trong quá trình chuẩn bị project, tác giả sử dụng ChatGPT/Codex để hỗ trợ rà soát mã nguồn, tổ chức thí nghiệm, trực quan hóa và hiệu đính ngôn ngữ. Tác giả đã kiểm tra mã, nghiệm, số liệu và chịu trách nhiệm cuối cùng về nội dung báo cáo.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence, pp. 399–404.

[2] Google, a. Cp-sat solver. `https://developers.google.com/optimization/cp/cp_solver`. OR-Tools documentation, truy cập ngày 11-10-2026.

[3] Google, b. The n-queens problem. `https://developers.google.com/optimization/cp/queens`. OR-Tools documentation, truy cập ngày 11-10-2026.

[4] Gurobi Optimization, LLC, 2026. Gurobi optimizer reference manual, version 13.0. `https://docs.gurobi.com/projects/optimizer/en/current/`.

[5] IBM, . Ibm ilog cp optimizer overview. `https://www.ibm.com/docs/en/icos/22.1.0?topic=cpco-overview`. Truy cập ngày 11-10-2026.

[6] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. Pysat: A python toolkit for prototyping with sat oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.

[7] PySAT developers, . Pysat documentation. `https://pysathq.github.io/docs/html/`. Truy cập ngày 11-10-2026.

[8] Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:`10.1007/11564751_73`.